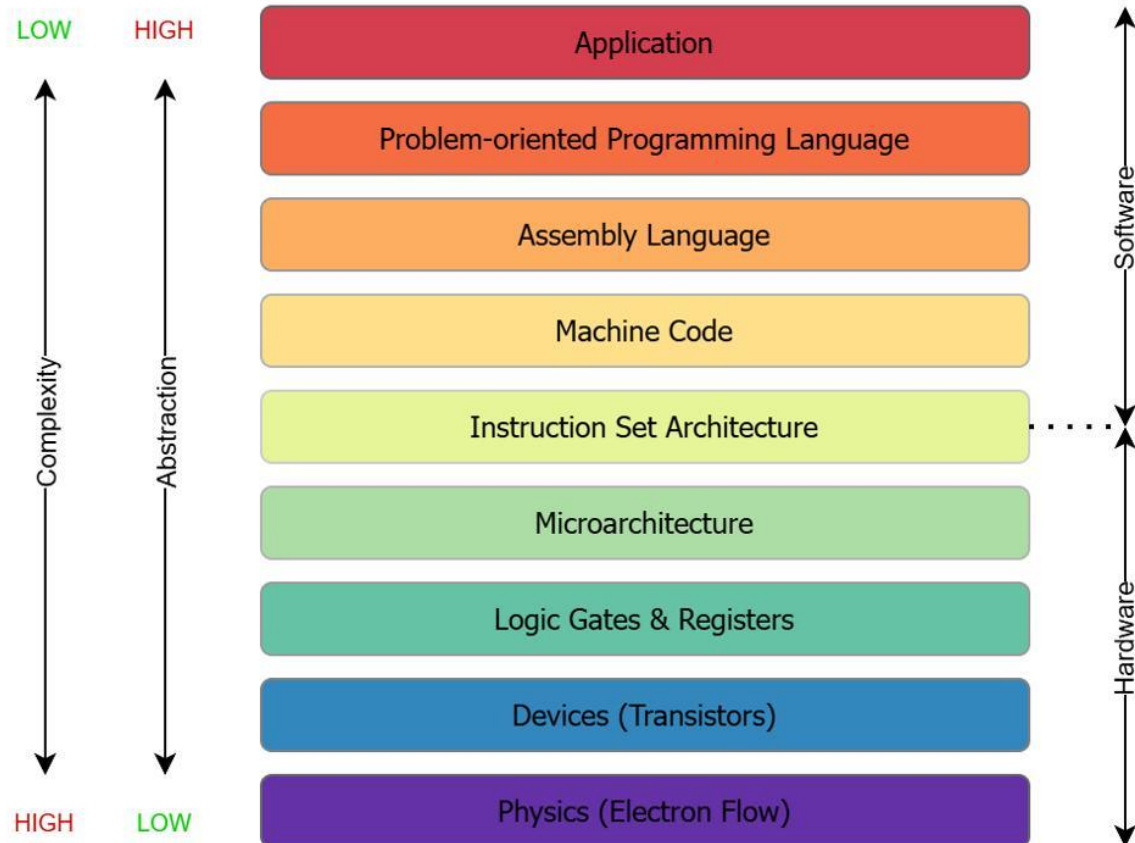


Chapter: Abstraction for Programming (5 Hours)

Layers of Abstraction



How Does Software Work?



USER INTERFACE

This is how the user communicates with the software, and where most differences exist.



PRESENTATION LAYER

This controls the UI layer, sending the information to show you the right thing.



BUSINESS LOGIC LAYER

This is where decisions are made, knowing what to do when the user does something.



DATA LOGIC LAYER

This is where the system performs tasks to retrieve or manipulate the data as needed.



DATA LAYER

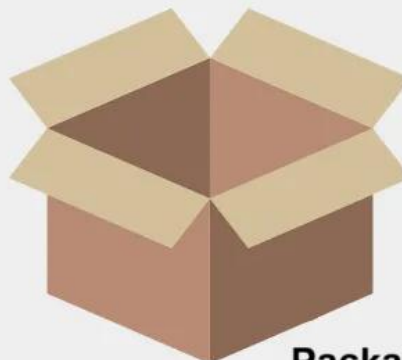
This is where the actual information is stored so your computer 'remembers'.



API



Libraries



Package



Documentation

Learning Objectives

After studying this chapter, students will be able to:

- Define **abstraction** in programming.
 - Explain different **levels of abstraction** in computer systems.
 - Understand the role of **libraries, system software, and toolkits**.
 - Describe the importance of **high-level programming languages**.
 - Explain the concepts of **Object-Oriented Programming (OOP)**.
 - Compare abstraction mechanisms used in modern software development.
 - Apply abstraction principles in software engineering and multimedia application development.
-

1. Introduction to Abstraction

What is Abstraction?

Abstraction is the process of hiding unnecessary implementation details while exposing only the essential features needed to perform a task.

It enables programmers to solve complex problems without understanding every low-level hardware or software operation.

Definition

Abstraction is a software engineering principle that simplifies system design by hiding implementation details and exposing only the necessary functionality through well-defined interfaces.

Simple Real-Life Example

Driving a Car

When driving a car, the driver uses:

- Steering wheel
- Accelerator
- Brake
- Gear

The driver **does not need to know**:

- How fuel is injected
- How pistons move
- How the engine control unit works

The car provides an **abstraction** of a very complex mechanical system.

ATM Machine

When withdrawing money:

You only:

- Insert the ATM card
- Enter the PIN
- Select the amount

You do **not** see:

- Database queries
- Banking servers
- Encryption algorithms
- Network communication

The ATM hides these implementation details.

Programming Example

Instead of writing instructions directly to hardware, a programmer can use a library function:

```
#include <iostream>
using namespace std;

int main()
{
    cout << "Hello World";
}
```

The programmer does **not** need to know:

- How characters are stored in memory.
- How the operating system writes to the screen.

- How the graphics driver controls the monitor.

The library and operating system provide the abstraction.

Why is Abstraction Important?

Abstraction helps programmers:

- Reduce complexity.
 - Improve code readability.
 - Increase productivity.
 - Reuse existing components.
 - Simplify maintenance.
 - Develop large-scale software.
 - Enhance software portability.
-

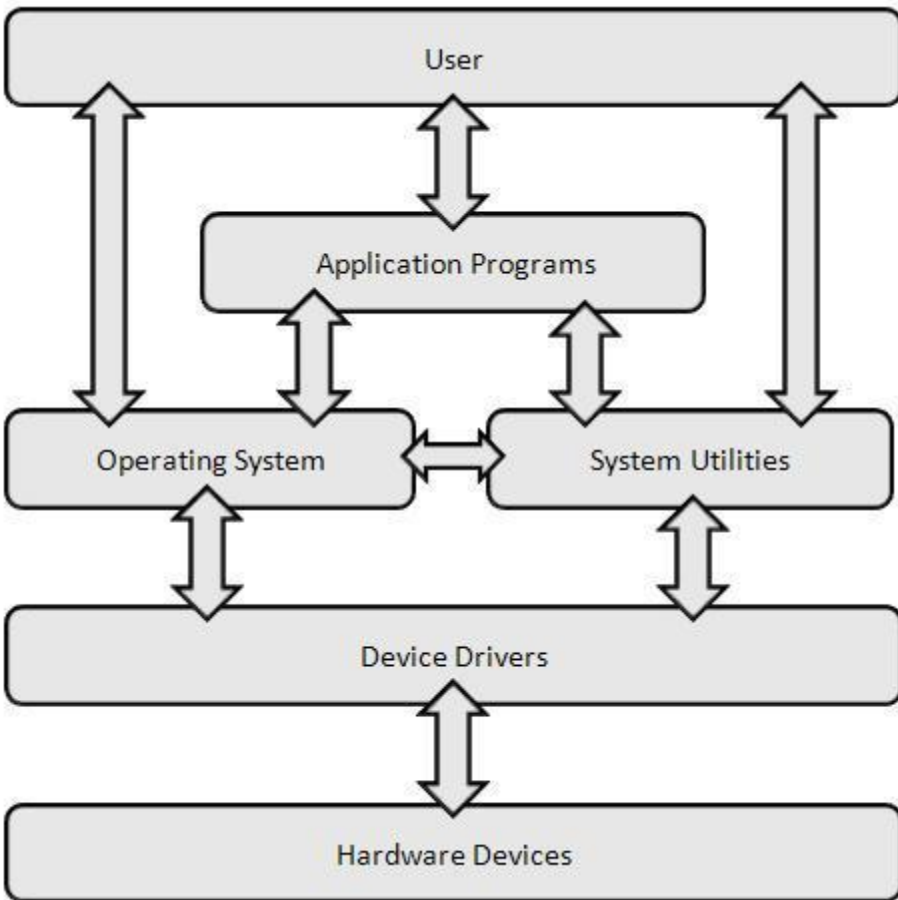
Advantages of Abstraction

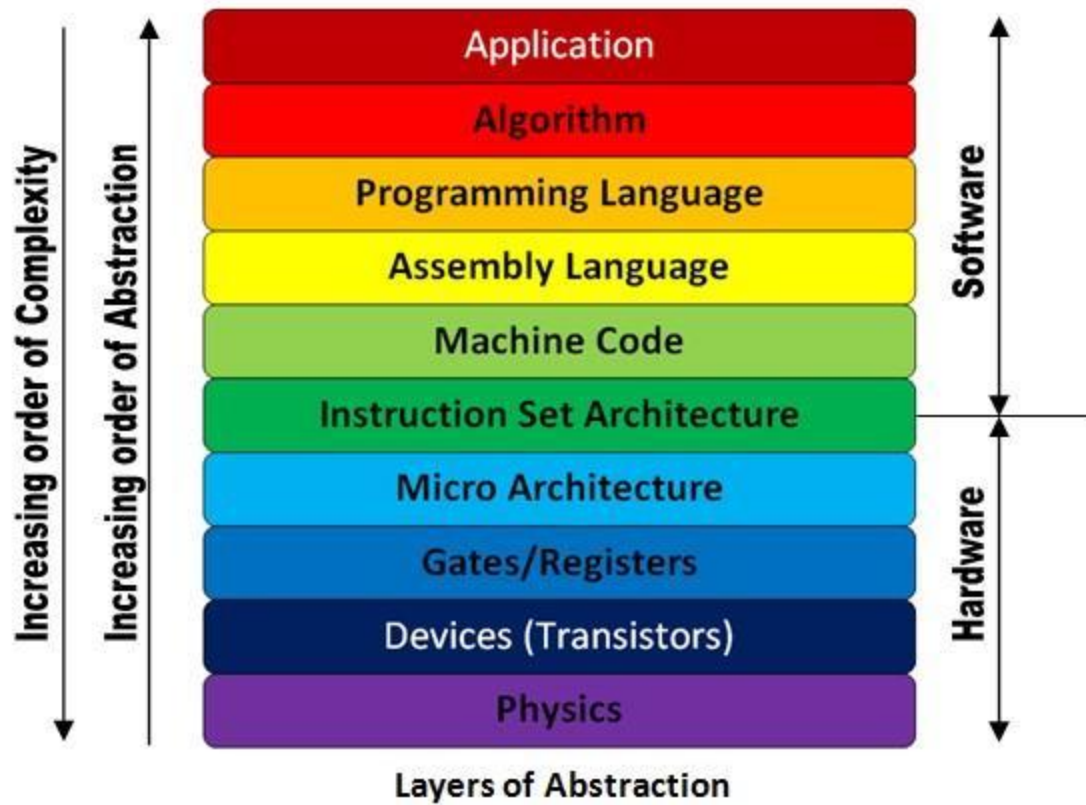
Advantage	Description
Simplicity	Reduces system complexity
Reusability	Reuses existing components
Maintainability	Easier to modify and debug
Security	Hides sensitive implementation details
Portability	Runs on different hardware platforms
Productivity	Faster software development

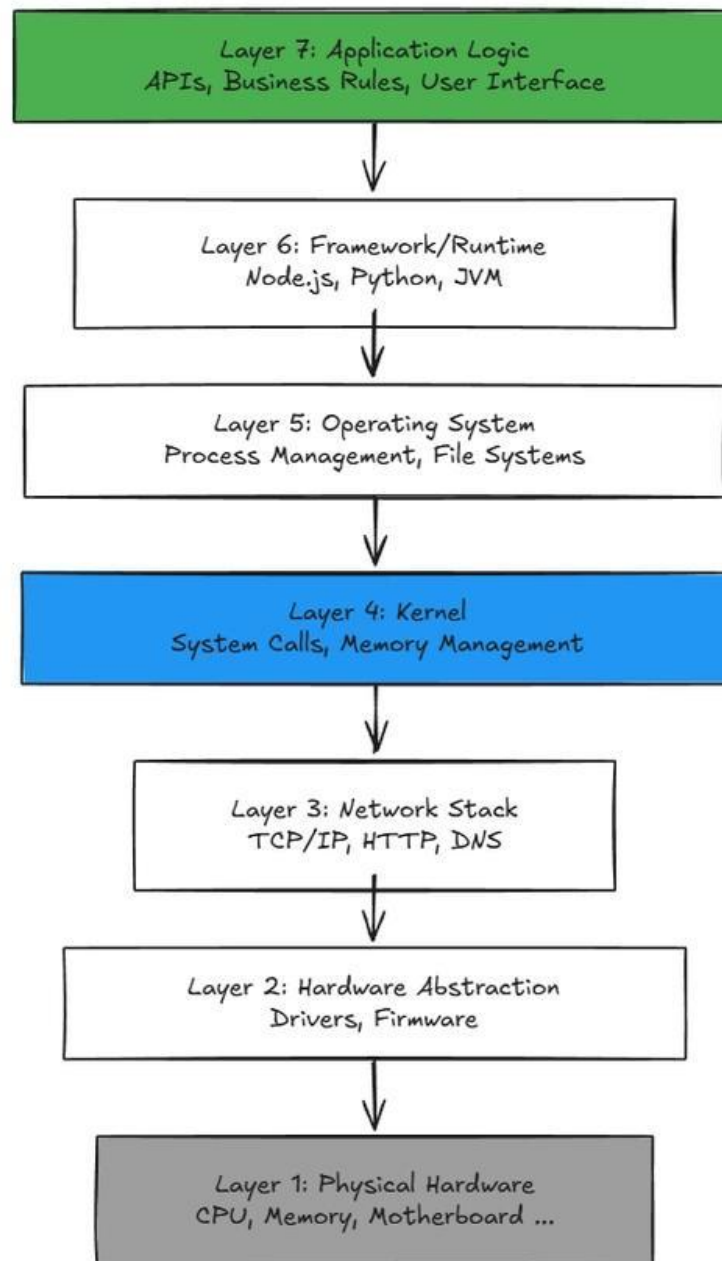
Disadvantages

- Slight performance overhead.
 - Hidden implementation can make debugging more difficult.
 - Improper abstraction can reduce flexibility.
-

2. Abstraction Levels







6

Programming is organized into multiple layers. Each layer hides the complexity of the layer below it.

```
+-----+  
| User Applications |
```

-----	-----
Programming Libraries	
-----	-----
Programming Languages	
-----	-----
Operating System	
-----	-----
Device Drivers	
-----	-----
Computer Hardware	
-----	-----

Level 1: Hardware Level

The lowest abstraction level.

Includes:

- CPU
- Memory (RAM)
- Hard Disk/SSD
- GPU
- Keyboard
- Mouse
- Network Interface Card

Example

CPU executes binary instructions such as:

10110010

Programming directly at this level is difficult.

Level 2: Machine Language

Machine language consists entirely of binary instructions.

Example

10110000 01100001

Characteristics

- Fastest execution
 - Hardware dependent
 - Difficult to write
 - Difficult to debug
-

Level 3: Assembly Language

Assembly language uses mnemonic instructions.

Example

```
MOV AX, 10  
ADD AX, 5
```

Advantages

- Easier than machine language
 - Efficient
 - Hardware dependent
-

Level 4: High-Level Languages

Examples

- C
- C++
- Java
- Python
- C#
- JavaScript

Example

```
print("Hello")
```

Advantages

- Easy to understand
- Portable
- Faster development
- Large library support

Level 5: Application Level

Applications include:

- MS Word
- Photoshop
- VLC Media Player
- Web Browsers
- Multimedia Editors

Users interact only with this highest level.

Comparison of Abstraction Levels

Level	Users	Complexity	Examples
Hardware	Engineers	Very High	CPU, RAM
Machine Language	System Programmers	Very High	Binary instructions
Assembly	Embedded Developers	High	MOV, ADD
High-Level Language	Software Developers	Moderate	Python, Java
Application	End Users	Very Low	MS Word, VLC

3. Libraries

Definition

A **library** is a collection of pre-written, reusable functions, classes, or modules that programmers can use without writing everything from scratch.

Libraries increase productivity and reduce code duplication.

Types of Libraries

Standard Libraries

Provided with programming languages.

Examples:

- C Standard Library (`stdio.h`, `math.h`)
 - Java Standard Library
 - Python Standard Library
-

Third-Party Libraries

Developed by external organizations or communities.

Examples:

- OpenCV (Computer Vision)
 - NumPy (Scientific Computing)
 - TensorFlow (Machine Learning)
 - Pygame (Game Development)
-

Example (C)

```
#include <math.h>

double value = sqrt(25);
```

The programmer calls `sqrt()` without implementing the square-root algorithm.

Multimedia Libraries

Examples:

- OpenCV
- SDL (Simple DirectMedia Layer)
- OpenGL
- DirectX
- FFmpeg

These provide ready-made functions for graphics, video, and audio processing.

Advantages of Libraries

- Save development time.
 - Improve reliability.
 - Reuse tested code.
 - Simplify complex tasks.
-

4. System Software

Definition

System software manages computer hardware and provides a platform for application software.

It acts as an intermediary between users, applications, and hardware.

Types of System Software

Operating System

Examples:

- Windows
- Linux
- macOS
- Android

Functions:

- Process management
 - Memory management
 - File management
 - Device management
 - Security
-

Device Drivers

Allow the operating system to communicate with hardware devices.

Examples:

- Printer driver
 - Graphics driver
 - Audio driver
-

Language Translators

Convert source code into machine code.

Types:

- Compiler
 - Interpreter
 - Assembler
-

Utility Software

Examples:

- Antivirus
 - Disk Cleanup
 - Backup software
 - Compression utilities
-

System Software Architecture

```
User
|
Application Software
|
Operating System
|
Device Drivers
|
Hardware
```

Importance of System Software

- Controls hardware resources.
 - Provides user interfaces.
 - Executes application programs.
 - Ensures system security and stability.
-

5. Toolkits

Definition

A **toolkit** is a collection of software tools, libraries, APIs, documentation, and utilities designed to help developers build specific types of applications.

Unlike a simple library, a toolkit usually includes multiple integrated resources.

Components of a Toolkit

- Libraries
 - APIs
 - Compilers
 - Debuggers
 - Documentation
 - Templates
 - Sample projects
-

Examples

Multimedia Toolkit

- OpenCV
- SDL
- Qt Multimedia
- FFmpeg

Mobile Development Toolkit

- Android SDK
- Flutter SDK

Game Development Toolkit

- Unity
 - Unreal Engine
-

Advantages

- Faster development.
 - Better integration.
 - Cross-platform support.
 - Rich documentation.
-

6. Higher Programming Languages

Definition

High-level programming languages allow programmers to write code using syntax that is close to human language rather than machine instructions.

Characteristics

- Easy to read.
 - Easy to write.
 - Portable.
 - Structured.
 - Supports modular programming.
 - Supports object-oriented programming.
-

Examples

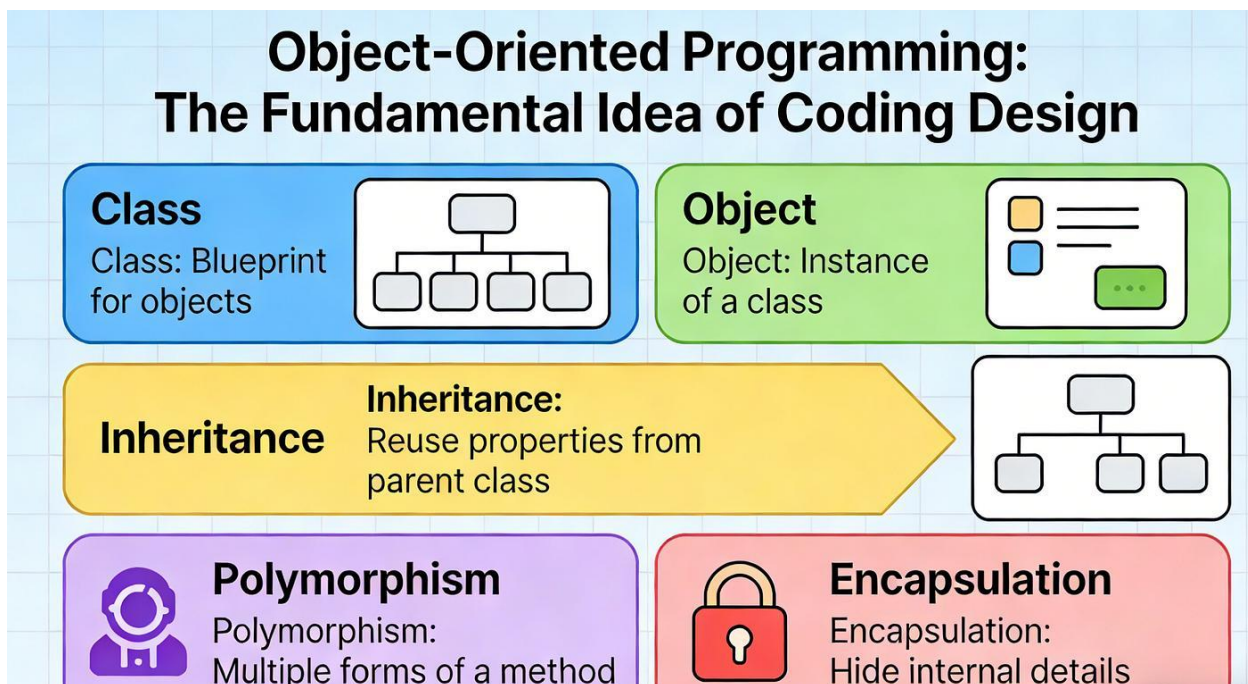
Language	Common Applications
C	System programming
C++	Games, multimedia
Java	Enterprise software

Language	Common Applications
Python	AI, ML, multimedia
C#	Desktop and game development
JavaScript	Web development
PHP	Web applications

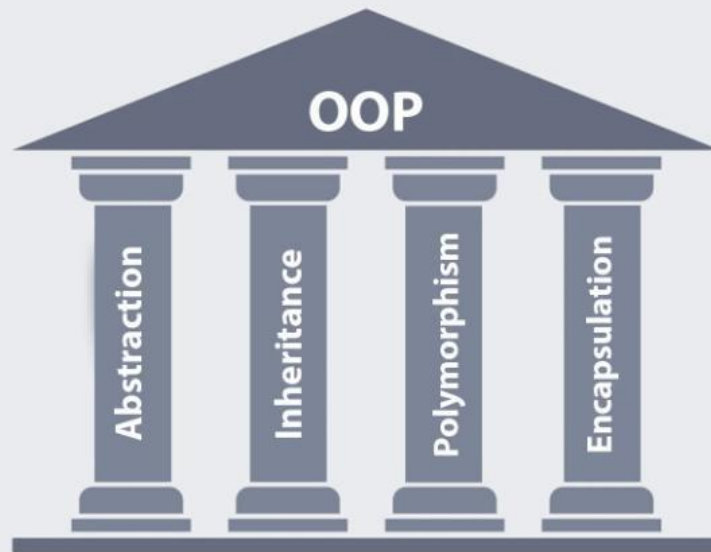
Comparison

Feature	Low-Level	High-Level
Readability	Low	High
Portability	Low	High
Development Speed	Slow	Fast
Hardware Dependency	High	Low

7. Object-Oriented Approaches (OOP)



The Four Pillars



COMPONENTS OF OBJECT-ORIENTED PROGRAMMING (OOP)

OBJECTS



```
Car myCar = new Car();
```

CLASSES



```
class Car {  
    String color;  
    void drive() {}  
}
```

ENCAPSULATION



```
private int balance;  
public int getBalance(){  
    return balance;  
}  
public void setBalance(int  
    b) balance = b; }
```

INHERITANCE



```
class Dog extends Animal {
```

POLYMORPHISM



```
animal.makeSound();
```

ABSTRACTION



```
abstract class Shape {  
    abstract void draw();  
}
```

Object-Oriented Programming (OOP) is a programming paradigm that organizes software into **objects**, which combine **data (attributes)** and **methods (functions)**.

Objects represent real-world entities.

Example

Class

`Student`

Object

`Ram`
`Hari`
`Sita`

Each student object has:

Attributes:

- Name
- Roll Number
- Address

Methods:

- `Study()`
 - `AttendClass()`
 - `SubmitAssignment()`
-

Four Main Principles of OOP

1. Encapsulation

Combines data and methods into a single unit (class) and restricts direct access to internal data.

Example (Java)

```
class Student {  
    private String name;
```

```
public void setName(String n){
    name = n;
}

public String getName(){
    return name;
}
}
```

Advantages

- Data protection
 - Improved security
 - Easier maintenance
-

2. Abstraction

Shows only essential features and hides implementation details.

Example:

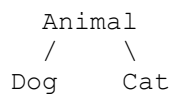
```
car.start();
```

The user does not need to know how the engine starts internally.

3. Inheritance

Allows a class to inherit properties and methods from another class.

Example



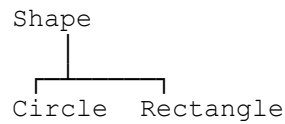
Benefits:

- Code reuse
 - Easier maintenance
 - Reduced duplication
-

4. Polymorphism

Allows one interface to represent multiple implementations.

Example



Both implement:

`draw()`

but each provides its own behavior.

Benefits of OOP

- Code reuse
 - Modularity
 - Scalability
 - Easier debugging
 - Better maintainability
 - Improved software design
-

Applications of OOP

- Multimedia software
 - Game development
 - Mobile applications
 - Banking systems
 - E-commerce platforms
 - Artificial Intelligence
 - Healthcare systems
-

Summary

Topic	Key Points
Abstraction	Hides implementation details and exposes essential features
Abstraction Levels	Hardware → Machine Language → Assembly → High-Level Languages → Applications
Libraries	Reusable collections of functions and classes
System Software	Operating systems, drivers, translators, and utilities
Toolkits	Integrated collections of tools, libraries, APIs, and documentation
High-Level Languages	Human-readable languages such as C, C++, Java, and Python
Object-Oriented Programming	Organizes programs using classes and objects
OOP Principles	Encapsulation, Abstraction, Inheritance, and Polymorphism

Advantages of Programming Abstraction

- Simplifies software development.
- Encourages code reuse.
- Improves readability and maintainability.
- Reduces development time.
- Enables portability across platforms.
- Supports large-scale software engineering.